# DEVOPS TOOLS & OPERATIONS GUIDE

## AI-Based Monitoring Solution for Oil & Gas Industry

**Document Version**: 1.0 **Date**: December 2025 **Audience**: DevOps Engineers, SREs, Platform Engineers

---

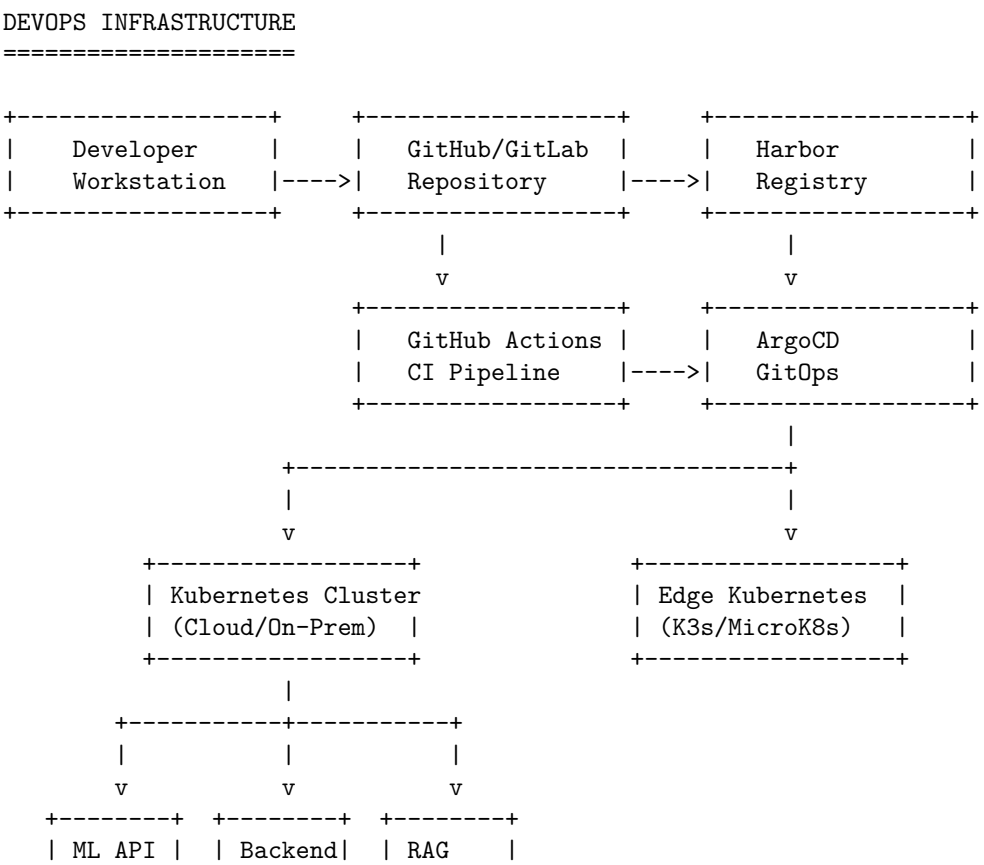
# TABLE OF CONTENTS


1. DevOps Environment Overview
2. Docker Operations
3. Kubernetes Operations
4. Helm Chart Management
5. CI/CD with GitHub Actions
6. ArgoCD GitOps
7. Harbor Registry Operations
8. Secrets Management with Vault
9. Infrastructure as Code
10. Complete DevOps Workflows


---

# 1. DEVOPS ENVIRONMENT OVERVIEW

## 1.1 Infrastructure Architecture



```
   | Pods   |  | Pods   |  | Pods   |
   +--------+  +--------+  +--------+
```

### 1.2 Tool Versions & Installation

```bash
# Docker
curl -fsSL https://get.docker.com | sh
docker --version  # 24.0+

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/
chmod +x kubectl && sudo mv kubectl /usr/local/bin/
kubectl version --client  # 1.28+

# Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version  # 3.13+

# ArgoCD CLI
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linu
chmod +x argocd-linux-amd64 && sudo mv argocd-linux-amd64 /usr/local/bin/argocd
argocd version --client

# Vault CLI
wget https://releases.hashicorp.com/vault/1.15.0/vault_1.15.0_linux_amd64.zip
unzip vault_1.15.0_linux_amd64.zip && sudo mv vault /usr/local/bin/
vault version

# k9s (Kubernetes TUI)
wget https://github.com/derailed/k9s/releases/latest/download/k9s_Linux_amd64.tar.gz
tar xzf k9s_Linux_amd64.tar.gz && sudo mv k9s /usr/local/bin/
```

### 1.3 Environment Configuration

```bash
# ~/.devops-env
# Source this file: source ~/.devops-env

# Kubernetes
export KUBECONFIG=~/.kube/config
export KUBE_CONTEXT="oilgas-production"

# Docker Registry
export DOCKER_REGISTRY="harbor.oilgas.internal"
export DOCKER_NAMESPACE="oilgas"

# ArgoCD
export ARGOCD_SERVER="argocd.oilgas.internal"
export ARGOCD_AUTH_TOKEN="your-argocd-token"

# Vault
export VAULT_ADDR="https://vault.oilgas.internal:8200"
export VAULT_TOKEN="your-vault-token"

# GitHub
```

```
export GITHUB_TOKEN="your-github-token"
export GITHUB_REPOSITORY="oilgas/ai-monitoring"
```

---

# 2. DOCKER OPERATIONS

## 2.1 Docker CLI Commands

```
# ============================================
# IMAGE MANAGEMENT
# ============================================

# Build image
docker build -t oilgas/ml-api:v1.0.0 .
docker build -f Dockerfile.gpu -t oilgas/ml-api:v1.0.0-gpu .

# Build with build args
docker build \
    --build-arg PYTHON_VERSION=3.10 \
    --build-arg TORCH_VERSION=2.1.0 \
    -t oilgas/ml-api:v1.0.0 .

# Multi-platform build
docker buildx build \
    --platform linux/amd64,linux/arm64 \
    -t oilgas/ml-api:v1.0.0 \
    --push .

# List images
docker images
docker images --filter "reference=oilgas/*"

# Tag image
docker tag oilgas/ml-api:v1.0.0 harbor.oilgas.internal/oilgas/ml-api:v1.0.0

# Push to registry
docker push harbor.oilgas.internal/oilgas/ml-api:v1.0.0

# Pull image
docker pull harbor.oilgas.internal/oilgas/ml-api:v1.0.0

# Remove image
docker rmi oilgas/ml-api:v1.0.0
docker image prune -a  # Remove unused images

# ============================================
# CONTAINER MANAGEMENT
# ============================================

# Run container
docker run -d \
    --name ml-api \
    --gpus all \
```

```
    -p 8000:8000 \
    -v /data/models:/models:ro \
    -e MODEL_PATH=/models/anomaly.pt \
    oilgas/ml-api:v1.0.0

# Run interactive
docker run -it --rm oilgas/ml-api:v1.0.0 /bin/bash

# List containers
docker ps
docker ps -a  # Include stopped

# Container logs
docker logs ml-api
docker logs -f ml-api  # Follow
docker logs --tail 100 ml-api

# Execute command in container
docker exec -it ml-api /bin/bash
docker exec ml-api python -c "import torch; print(torch.cuda.is_available())"

# Container stats
docker stats ml-api

# Stop/Start/Restart
docker stop ml-api
docker start ml-api
docker restart ml-api

# Remove container
docker rm ml-api
docker rm -f ml-api  # Force

# ============================================
# DOCKER COMPOSE
# ============================================

# Start services
docker-compose up -d
docker-compose -f docker-compose.prod.yml up -d

# Scale service
docker-compose up -d --scale ml-api=3

# View logs
docker-compose logs -f ml-api

# Stop services
docker-compose down
docker-compose down -v  # Remove volumes

# Rebuild and restart
docker-compose up -d --build ml-api
```

### 2.2 Dockerfile Examples

```
# Dockerfile.ml-api
# Multi-stage build for ML API

# Stage 1: Builder
FROM python:3.10-slim as builder

WORKDIR /build

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

# Stage 2: Runtime
FROM nvidia/cuda:12.1-runtime-ubuntu22.04

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app

WORKDIR /app

# Install Python
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.10 \
    python3-pip \
    && rm -rf /var/lib/apt/lists/*

# Copy wheels and install
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels

# Copy application
COPY src/ ./src/
COPY models/ ./models/
COPY configs/ ./configs/

# Create non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

EXPOSE 8000

CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```dockerfile
# Dockerfile.edge
# Optimized for Jetson edge devices

FROM nvcr.io/nvidia/l4t-pytorch:r35.2.1-pth2.0-py3

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# Install dependencies
COPY requirements-edge.txt .
RUN pip3 install --no-cache-dir -r requirements-edge.txt

# Copy TensorRT models
COPY models/tensorrt/ ./models/

# Copy inference code
COPY src/inference/ ./src/inference/

# Run as non-root
RUN useradd -m -u 1000 edge && chown -R edge:edge /app
USER edge

EXPOSE 8080

CMD ["python3", "src/inference/edge_server.py"]
```

### 2.3 Docker Compose Configuration

```yaml
# docker-compose.yml
version: '3.8'

services:
  ml-api:
    build:
      context: .
      dockerfile: Dockerfile.ml-api
    image: harbor.oilgas.internal/oilgas/ml-api:${VERSION:-latest}
    container_name: ml-api
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      - MODEL_PATH=/models/anomaly_detector.pt
      - DEVICE=cuda
      - LOG_LEVEL=INFO
    volumes:
      - ./models:/models:ro
      - ./logs:/app/logs
    deploy:
      resources:
        reservations:
          devices:
```

```
          - driver: nvidia
            count: 1
            capabilities: [gpu]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    interval: 30s
    timeout: 10s
    retries: 3
  networks:
    - oilgas-network

backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  image: harbor.oilgas.internal/oilgas/backend:${VERSION:-latest}
  container_name: backend
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - DATABASE_URL=postgresql://user:pass@postgres:5432/oilgas
    - REDIS_URL=redis://redis:6379
    - ML_API_URL=http://ml-api:8000
  depends_on:
    - postgres
    - redis
    - ml-api
  networks:
    - oilgas-network

postgres:
  image: postgres:16-alpine
  container_name: postgres
  restart: unless-stopped
  environment:
    - POSTGRES_USER=oilgas
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DB=oilgas_ai
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U oilgas"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - oilgas-network

redis:
  image: redis:7-alpine
  container_name: redis
  restart: unless-stopped
```

```
    command: redis-server --requirepass ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    networks:
      - oilgas-network

volumes:
  postgres_data:
  redis_data:

networks:
  oilgas-network:
    driver: bridge
```

---

# 3. KUBERNETES OPERATIONS

## 3.1 kubectl Commands

```
# ==========================================
# CLUSTER MANAGEMENT
# ==========================================

# Context management
kubectl config get-contexts
kubectl config use-context oilgas-production
kubectl config current-context

# Cluster info
kubectl cluster-info
kubectl get nodes
kubectl describe node <node-name>

# Namespace operations
kubectl get namespaces
kubectl create namespace oilgas-ml
kubectl config set-context --current --namespace=oilgas-ml

# ==========================================
# WORKLOAD MANAGEMENT
# ==========================================

# Deployments
kubectl get deployments
kubectl get deployment ml-api -o yaml
kubectl describe deployment ml-api
kubectl rollout status deployment/ml-api
kubectl rollout history deployment/ml-api
kubectl rollout undo deployment/ml-api
kubectl rollout restart deployment/ml-api

# Scale deployment
kubectl scale deployment ml-api --replicas=3
```

```
kubectl autoscale deployment ml-api --min=2 --max=10 --cpu-percent=80

# Pods
kubectl get pods
kubectl get pods -o wide
kubectl get pods -l app=ml-api
kubectl describe pod ml-api-xxx
kubectl logs ml-api-xxx
kubectl logs -f ml-api-xxx --tail=100
kubectl logs ml-api-xxx -c init-container  # Specific container
kubectl exec -it ml-api-xxx -- /bin/bash
kubectl port-forward pod/ml-api-xxx 8000:8000

# Services
kubectl get services
kubectl get svc ml-api -o yaml
kubectl describe service ml-api
kubectl expose deployment ml-api --port=8000 --type=ClusterIP

# ConfigMaps & Secrets
kubectl get configmaps
kubectl get secrets
kubectl create configmap app-config --from-file=config.yaml
kubectl create secret generic db-credentials --from-literal=password=secret
kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 -d

# ==========================================
# RESOURCE OPERATIONS
# ==========================================

# Apply manifest
kubectl apply -f deployment.yaml
kubectl apply -f k8s/ --recursive

# Delete resources
kubectl delete -f deployment.yaml
kubectl delete deployment ml-api
kubectl delete pod ml-api-xxx --force --grace-period=0

# Resource usage
kubectl top nodes
kubectl top pods
kubectl describe resourcequota

# Events
kubectl get events --sort-by='.lastTimestamp'
kubectl get events -w  # Watch

# ==========================================
# DEBUGGING
# ==========================================

# Debug pod
kubectl debug ml-api-xxx -it --image=busybox
```

```
# Run temporary pod
kubectl run test-pod --rm -it --image=curlimages/curl -- sh

# Copy files
kubectl cp ml-api-xxx:/app/logs ./logs
kubectl cp ./config.yaml ml-api-xxx:/app/config.yaml
```

### 3.2 Kubernetes Manifests

```
# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: oilgas-ml
  labels:
    name: oilgas-ml
---
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: ml-api-config
  namespace: oilgas-ml
data:
  MODEL_PATH: "/models/anomaly_detector.pt"
  LOG_LEVEL: "INFO"
  DEVICE: "cuda"
  config.yaml: |
    model:
      name: AnomalyAutoencoder
      threshold: 0.5
    server:
      host: 0.0.0.0
      port: 8000
---
# k8s/secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: ml-api-secrets
  namespace: oilgas-ml
type: Opaque
stringData:
  DATABASE_URL: "postgresql://user:password@postgres:5432/oilgas"
  MLFLOW_TRACKING_TOKEN: "mlflow-token"
---
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-api
  namespace: oilgas-ml
  labels:
```

```yaml
    app: ml-api
    version: v1.0.0
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ml-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: ml-api
        version: v1.0.0
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8000"
        prometheus.io/path: "/metrics"
    spec:
      serviceAccountName: ml-api
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        fsGroup: 1000
      containers:
        - name: ml-api
          image: harbor.oilgas.internal/oilgas/ml-api:v1.0.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8000
              protocol: TCP
          envFrom:
            - configMapRef:
                name: ml-api-config
            - secretRef:
                name: ml-api-secrets
          resources:
            requests:
              cpu: "500m"
              memory: "1Gi"
              nvidia.com/gpu: "1"
            limits:
              cpu: "2000m"
              memory: "4Gi"
              nvidia.com/gpu: "1"
          volumeMounts:
            - name: models
              mountPath: /models
              readOnly: true
            - name: config
              mountPath: /app/config
```

```yaml
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 30
            periodSeconds: 10
            timeoutSeconds: 5
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /ready
              port: 8000
            initialDelaySeconds: 10
            periodSeconds: 5
            timeoutSeconds: 3
            failureThreshold: 3
      volumes:
        - name: models
          persistentVolumeClaim:
            claimName: ml-models-pvc
        - name: config
          configMap:
            name: ml-api-config
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: gpu-type
                    operator: In
                    values:
                      - nvidia-a100
                      - nvidia-v100
      tolerations:
        - key: "nvidia.com/gpu"
          operator: "Exists"
          effect: "NoSchedule"
---
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: ml-api
  namespace: oilgas-ml
  labels:
    app: ml-api
spec:
  type: ClusterIP
  selector:
    app: ml-api
  ports:
    - name: http
      port: 8000
      targetPort: 8000
```

```yaml
      protocol: TCP
---
# k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ml-api-ingress
  namespace: oilgas-ml
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - ml-api.oilgas.example.com
      secretName: ml-api-tls
  rules:
    - host: ml-api.oilgas.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: ml-api
                port:
                  number: 8000
---
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ml-api-hpa
  namespace: oilgas-ml
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ml-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
```

```
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 10
          periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15
---
# k8s/pdb.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: ml-api-pdb
  namespace: oilgas-ml
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: ml-api
```

---

# 4. HELM CHART MANAGEMENT

## 4.1 Helm CLI Commands

```
# ==========================================
# REPOSITORY MANAGEMENT
# ==========================================

# Add repositories
helm repo add stable https://charts.helm.sh/stable
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add prometheus https://prometheus-community.github.io/helm-charts
helm repo add nvidia https://nvidia.github.io/gpu-operator

# Update repositories
helm repo update

# Search charts
helm search repo postgresql
helm search hub grafana

# ==========================================
# CHART INSTALLATION
# ==========================================
```

```bash
# Install chart
helm install postgresql bitnami/postgresql -n oilgas-db

# Install with values file
helm install ml-api ./charts/ml-api -f values.yaml -n oilgas-ml

# Install with set values
helm install ml-api ./charts/ml-api \
    --set image.tag=v1.0.0 \
    --set replicas=3 \
    --set resources.gpu=1 \
    -n oilgas-ml

# Install specific version
helm install prometheus prometheus/prometheus --version 15.0.0

# Dry run
helm install ml-api ./charts/ml-api --dry-run --debug

# ==========================================
# CHART MANAGEMENT
# ==========================================

# List releases
helm list -A
helm list -n oilgas-ml

# Get release info
helm status ml-api -n oilgas-ml
helm get values ml-api -n oilgas-ml
helm get manifest ml-api -n oilgas-ml

# Upgrade release
helm upgrade ml-api ./charts/ml-api -f values.yaml -n oilgas-ml
helm upgrade --install ml-api ./charts/ml-api -n oilgas-ml  # Install if not exists

# Rollback
helm rollback ml-api 1 -n oilgas-ml
helm history ml-api -n oilgas-ml

# Uninstall
helm uninstall ml-api -n oilgas-ml

# ==========================================
# CHART DEVELOPMENT
# ==========================================

# Create new chart
helm create ml-api

# Lint chart
helm lint ./charts/ml-api
```

```
# Template render
helm template ml-api ./charts/ml-api -f values.yaml

# Package chart
helm package ./charts/ml-api

# Push to registry
helm push ml-api-1.0.0.tgz oci://harbor.oilgas.internal/charts
```

### 4.2 Helm Chart Structure

```
charts/ml-api/
  Chart.yaml
  values.yaml
  values-dev.yaml
  values-prod.yaml
  templates/
      _helpers.tpl
      deployment.yaml
      service.yaml
      ingress.yaml
      configmap.yaml
      secret.yaml
      hpa.yaml
      pdb.yaml
      serviceaccount.yaml
      NOTES.txt
  charts/
```

```yaml
# Chart.yaml
apiVersion: v2
name: ml-api
description: ML API for Oil & Gas AI Monitoring
type: application
version: 1.0.0
appVersion: "1.0.0"
keywords:
  - ml
  - ai
  - inference
maintainers:
  - name: DevOps Team
    email: devops@oilgas.com
dependencies:
  - name: redis
    version: "17.x.x"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
```

```yaml
# values.yaml
replicaCount: 2

image:
  repository: harbor.oilgas.internal/oilgas/ml-api
  tag: "v1.0.0"
```

```yaml
  pullPolicy: Always

imagePullSecrets:
  - name: harbor-creds

serviceAccount:
  create: true
  name: ml-api

service:
  type: ClusterIP
  port: 8000

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
  hosts:
    - host: ml-api.oilgas.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: ml-api-tls
      hosts:
        - ml-api.oilgas.example.com

resources:
  requests:
    cpu: 500m
    memory: 1Gi
    nvidia.com/gpu: 1
  limits:
    cpu: 2000m
    memory: 4Gi
    nvidia.com/gpu: 1

autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
  targetMemoryUtilizationPercentage: 80

nodeSelector:
  gpu-type: nvidia-a100

tolerations:
  - key: "nvidia.com/gpu"
    operator: "Exists"
    effect: "NoSchedule"

affinity: {}
```

```yaml
config:
  model_path: /models/anomaly_detector.pt
  log_level: INFO
  device: cuda

secrets:
  database_url: ""
  mlflow_token: ""

persistence:
  enabled: true
  storageClass: "standard"
  size: 10Gi

redis:
  enabled: true
  auth:
    password: "redis-password"
```

```yaml
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "ml-api.fullname" . }}
  labels:
    {{- include "ml-api.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "ml-api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "ml-api.selectorLabels" . | nindent 8 }}
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      serviceAccountName: {{ include "ml-api.serviceAccountName" . }}
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: {{ .Values.service.port }}
          envFrom:
            - configMapRef:
                name: {{ include "ml-api.fullname" . }}-config
            - secretRef:
                name: {{ include "ml-api.fullname" . }}-secrets
          resources:
```

```
            {{- toYaml .Values.resources | nindent 12 }}
          livenessProbe:
            httpGet:
              path: /health
              port: {{ .Values.service.port }}
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /ready
              port: {{ .Values.service.port }}
            initialDelaySeconds: 10
            periodSeconds: 5
      {{- with .Values.nodeSelector }}
      nodeSelector:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.tolerations }}
      tolerations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
```

---

# 5. CI/CD WITH GITHUB ACTIONS

## 5.1 GitHub Actions Workflows

```yaml
# .github/workflows/ci.yaml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: harbor.oilgas.internal
  IMAGE_NAME: oilgas/ml-api

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Install linters
        run: pip install ruff black mypy
```

```yaml
    - name: Run ruff
      run: ruff check .

    - name: Run black
      run: black --check .

    - name: Run mypy
      run: mypy src/

test:
  runs-on: ubuntu-latest
  needs: lint
  services:
    postgres:
      image: postgres:16
      env:
        POSTGRES_PASSWORD: test
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

  steps:
    - uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.10'
        cache: 'pip'

    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install pytest pytest-cov pytest-asyncio

    - name: Run tests
      run: |
        pytest tests/ -v --cov=src --cov-report=xml

    - name: Upload coverage
      uses: codecov/codecov-action@v3
      with:
        files: coverage.xml

build:
  runs-on: ubuntu-latest
  needs: test
  permissions:
    contents: read
```

```yaml
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Harbor
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.HARBOR_USERNAME }}
          password: ${{ secrets.HARBOR_PASSWORD }}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=ref,event=branch
            type=sha,prefix=
            type=semver,pattern={{version}}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=registry,ref=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:buildcache
          cache-to: type=registry,ref=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:buildcache,mode=max

      - name: Scan image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
          format: 'sarif'
          output: 'trivy-results.sarif'

      - name: Upload scan results
        uses: github/codeql-action/upload-sarif@v2
        with:
          sarif_file: 'trivy-results.sarif'

  deploy-staging:
    runs-on: ubuntu-latest
    needs: build
    if: github.ref == 'refs/heads/develop'
    environment: staging
```

```yaml
    steps:
      - uses: actions/checkout@v4

      - name: Install ArgoCD CLI
        run: |
          curl -sSL -o /usr/local/bin/argocd https://github.com/argoproj/argo-cd/releases/latest/downloa
          chmod +x /usr/local/bin/argocd

      - name: Update image tag
        run: |
          cd k8s/overlays/staging
          kustomize edit set image ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
          git config user.name "GitHub Actions"
          git config user.email "actions@github.com"
          git add .
          git commit -m "Update staging image to ${{ github.sha }}"
          git push

      - name: Sync ArgoCD
        run: |
          argocd login ${{ secrets.ARGOCD_SERVER }} --token ${{ secrets.ARGOCD_TOKEN }} --grpc-web
          argocd app sync ml-api-staging --prune

  deploy-production:
    runs-on: ubuntu-latest
    needs: build
    if: github.ref == 'refs/heads/main'
    environment: production

    steps:
      - uses: actions/checkout@v4

      - name: Create Release PR
        uses: peter-evans/create-pull-request@v5
        with:
          title: "Release: Update production to ${{ github.sha }}"
          branch: release/${{ github.sha }}
          body: |
            ## Production Release

            - Image: `${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}`
            - Commit: ${{ github.sha }}

            **Checklist:**
            - [ ] Staging tests passed
            - [ ] Performance benchmarks acceptable
            - [ ] Rollback plan reviewed

# .github/workflows/release.yaml
name: Release Pipeline

on:
  push:
    tags:
```

```yaml
      - 'v*'

env:
  REGISTRY: harbor.oilgas.internal
  IMAGE_NAME: oilgas/ml-api

jobs:
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Get version
        id: version
        run: echo "VERSION=${GITHUB_REF#refs/tags/}" >> $GITHUB_OUTPUT

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Harbor
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.HARBOR_USERNAME }}
          password: ${{ secrets.HARBOR_PASSWORD }}

      - name: Build and push release
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ steps.version.outputs.VERSION }}
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest
          platforms: linux/amd64,linux/arm64

      - name: Install Helm
        uses: azure/setup-helm@v3

      - name: Package Helm chart
        run: |
          helm package charts/ml-api --version ${{ steps.version.outputs.VERSION }}

      - name: Push Helm chart
        run: |
          helm push ml-api-${{ steps.version.outputs.VERSION }}.tgz \
            oci://${{ env.REGISTRY }}/charts

      - name: Create GitHub Release
        uses: softprops/action-gh-release@v1
```

```yaml
        with:
          files: |
            ml-api-${{ steps.version.outputs.VERSION }}.tgz
          generate_release_notes: true
```

---

# 6. ARGOCD GITOPS

## 6.1 ArgoCD CLI Commands

```bash
# ==========================================
# AUTHENTICATION
# ==========================================

# Login
argocd login argocd.oilgas.internal --grpc-web
argocd login argocd.oilgas.internal --sso  # SSO login

# Get account info
argocd account get-user-info

# ==========================================
# APPLICATION MANAGEMENT
# ==========================================

# Create application
argocd app create ml-api \
    --repo https://github.com/oilgas/ai-monitoring.git \
    --path k8s/overlays/production \
    --dest-server https://kubernetes.default.svc \
    --dest-namespace oilgas-ml \
    --sync-policy automated \
    --auto-prune \
    --self-heal

# List applications
argocd app list

# Get application details
argocd app get ml-api
argocd app get ml-api -o yaml

# Sync application
argocd app sync ml-api
argocd app sync ml-api --prune  # Remove extra resources
argocd app sync ml-api --force  # Force sync

# Application actions
argocd app diff ml-api
argocd app rollback ml-api
argocd app delete ml-api

# History
```

```
argocd app history ml-api

# ==========================================
# PROJECT MANAGEMENT
# ==========================================

# Create project
argocd proj create oilgas \
    --description "Oil & Gas AI Monitoring" \
    --src https://github.com/oilgas/* \
    --dest https://kubernetes.default.svc,oilgas-*

# List projects
argocd proj list

# ==========================================
# CLUSTER MANAGEMENT
# ==========================================

# Add cluster
argocd cluster add oilgas-edge-cluster

# List clusters
argocd cluster list
```

### 6.2 ArgoCD Application Manifests

```yaml
# argocd/projects/oilgas.yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: oilgas
  namespace: argocd
spec:
  description: Oil & Gas AI Monitoring Project
  sourceRepos:
    - 'https://github.com/oilgas/*'
    - 'https://harbor.oilgas.internal/*'
  destinations:
    - namespace: 'oilgas-*'
      server: 'https://kubernetes.default.svc'
    - namespace: 'oilgas-*'
      server: 'https://edge-cluster.oilgas.internal'
  clusterResourceWhitelist:
    - group: ''
      kind: Namespace
  namespaceResourceBlacklist:
    - group: ''
      kind: ResourceQuota
  roles:
    - name: developer
      description: Developer role
      policies:
        - p, proj:oilgas:developer, applications, get, oilgas/*, allow
```

```
        - p, proj:oilgas:developer, applications, sync, oilgas/*, allow
      groups:
        - oilgas-developers
---
# argocd/applications/ml-api.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ml-api
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: oilgas
  source:
    repoURL: https://github.com/oilgas/ai-monitoring.git
    targetRevision: main
    path: charts/ml-api
    helm:
      valueFiles:
        - values.yaml
        - values-prod.yaml
      parameters:
        - name: image.tag
          value: v1.0.0
  destination:
    server: https://kubernetes.default.svc
    namespace: oilgas-ml
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false
    syncOptions:
      - CreateNamespace=true
      - PrunePropagationPolicy=foreground
      - PruneLast=true
    retry:
      limit: 5
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas
---
# argocd/applications/app-of-apps.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: oilgas-apps
```

```
  namespace: argocd
spec:
  project: oilgas
  source:
    repoURL: https://github.com/oilgas/ai-monitoring.git
    targetRevision: main
    path: argocd/applications
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

---

# 7. HARBOR REGISTRY OPERATIONS

## 7.1 Harbor CLI & API

```
# ============================================
# DOCKER CLI WITH HARBOR
# ============================================

# Login to Harbor
docker login harbor.oilgas.internal
# Enter username and password

# Push image
docker tag oilgas/ml-api:v1.0.0 harbor.oilgas.internal/oilgas/ml-api:v1.0.0
docker push harbor.oilgas.internal/oilgas/ml-api:v1.0.0

# Pull image
docker pull harbor.oilgas.internal/oilgas/ml-api:v1.0.0

# ============================================
# HARBOR API OPERATIONS
# ============================================

HARBOR_URL="https://harbor.oilgas.internal/api/v2.0"
HARBOR_AUTH="admin:Harbor12345"

# List projects
curl -u "$HARBOR_AUTH" "$HARBOR_URL/projects"

# List repositories in project
curl -u "$HARBOR_AUTH" "$HARBOR_URL/projects/oilgas/repositories"

# List tags for repository
curl -u "$HARBOR_AUTH" "$HARBOR_URL/projects/oilgas/repositories/ml-api/artifacts"

# Get vulnerability scan results
curl -u "$HARBOR_AUTH" "$HARBOR_URL/projects/oilgas/repositories/ml-api/artifacts/v1.0.0/additions/vuln
```

```
# Delete image tag
curl -X DELETE -u "$HARBOR_AUTH" "$HARBOR_URL/projects/oilgas/repositories/ml-api/artifacts/old-tag"

# Trigger garbage collection
curl -X POST -u "$HARBOR_AUTH" "$HARBOR_URL/system/gc/schedule" \
    -H "Content-Type: application/json" \
    -d '{"schedule":{"type":"Manual"}}'
```

### 7.2 Harbor Robot Accounts

```
# Create robot account via API
curl -X POST -u "$HARBOR_AUTH" "$HARBOR_URL/robots" \
    -H "Content-Type: application/json" \
    -d '{
        "name": "ci-robot",
        "description": "CI/CD robot account",
        "duration": -1,
        "level": "project",
        "permissions": [
            {
                "namespace": "oilgas",
                "kind": "project",
                "access": [
                    {"resource": "repository", "action": "pull"},
                    {"resource": "repository", "action": "push"},
                    {"resource": "artifact", "action": "delete"}
                ]
            }
        ]
    }'

# Use robot account
docker login harbor.oilgas.internal -u 'robot$ci-robot' -p '<robot-token>'
```

---

## 8. SECRETS MANAGEMENT WITH VAULT

### 8.1 Vault CLI Commands

```
# ============================================
# AUTHENTICATION
# ============================================

# Login with token
vault login

# Login with LDAP
vault login -method=ldap username=admin

# Login with Kubernetes
vault login -method=kubernetes role=oilgas-ml
```

```
# ==========================================
# SECRET OPERATIONS
# ==========================================

# Write secret
vault kv put secret/oilgas/ml-api \
    database_url="postgresql://user:pass@postgres:5432/db" \
    mlflow_token="token123"

# Read secret
vault kv get secret/oilgas/ml-api
vault kv get -format=json secret/oilgas/ml-api
vault kv get -field=database_url secret/oilgas/ml-api

# List secrets
vault kv list secret/oilgas/

# Delete secret
vault kv delete secret/oilgas/ml-api

# Secret versions
vault kv metadata get secret/oilgas/ml-api
vault kv get -version=1 secret/oilgas/ml-api

# ==========================================
# POLICY MANAGEMENT
# ==========================================

# Create policy
vault policy write ml-api-policy - <<EOF
path "secret/data/oilgas/ml-api" {
  capabilities = ["read"]
}
path "secret/data/oilgas/shared/*" {
  capabilities = ["read"]
}
EOF

# List policies
vault policy list
vault policy read ml-api-policy

# ==========================================
# KUBERNETES AUTH
# ==========================================

# Configure Kubernetes auth
vault auth enable kubernetes

vault write auth/kubernetes/config \
    kubernetes_host="https://kubernetes.default.svc:443" \
    kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt

# Create role for ML API
```

```
vault write auth/kubernetes/role/ml-api \
    bound_service_account_names=ml-api \
    bound_service_account_namespaces=oilgas-ml \
    policies=ml-api-policy \
    ttl=1h
```

### 8.2 Vault Agent Injector

```yaml
# k8s/deployment-with-vault.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-api
spec:
  template:
    metadata:
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "ml-api"
        vault.hashicorp.com/agent-inject-secret-config: "secret/data/oilgas/ml-api"
        vault.hashicorp.com/agent-inject-template-config: |
          {{- with secret "secret/data/oilgas/ml-api" -}}
          DATABASE_URL={{ .Data.data.database_url }}
          MLFLOW_TOKEN={{ .Data.data.mlflow_token }}
          {{- end }}
    spec:
      serviceAccountName: ml-api
      containers:
        - name: ml-api
          image: harbor.oilgas.internal/oilgas/ml-api:v1.0.0
          command:
            - /bin/sh
            - -c
            - |
              source /vault/secrets/config
              exec uvicorn src.main:app --host 0.0.0.0 --port 8000
```

---

## 9. INFRASTRUCTURE AS CODE

### 9.1 Kustomize

```yaml
# k8s/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: oilgas-ml

resources:
  - namespace.yaml
  - deployment.yaml
  - service.yaml
  - configmap.yaml
```

```yaml
  - hpa.yaml

commonLabels:
  app.kubernetes.io/name: ml-api
  app.kubernetes.io/part-of: oilgas-monitoring
---
# k8s/overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: oilgas-ml-prod

resources:
  - ../../base

namePrefix: prod-

commonLabels:
  environment: production

images:
  - name: harbor.oilgas.internal/oilgas/ml-api
    newTag: v1.0.0

replicas:
  - name: ml-api
    count: 5

patches:
  - path: patches/deployment-resources.yaml
  - path: patches/hpa.yaml

configMapGenerator:
  - name: ml-api-config
    behavior: merge
    literals:
      - LOG_LEVEL=WARNING
      - DEVICE=cuda

secretGenerator:
  - name: ml-api-secrets
    files:
      - secrets/database-url
```

```
# Kustomize commands
kustomize build k8s/overlays/production
kustomize build k8s/overlays/production | kubectl apply -f -
kubectl apply -k k8s/overlays/production
```

---

# 10. COMPLETE DEVOPS WORKFLOWS

## 10.1 Full Deployment Script

```bash
#!/bin/bash
# workflows/deploy.sh
# Complete deployment workflow

set -e

VERSION="${1:-latest}"
ENVIRONMENT="${2:-staging}"

echo "Deploying version $VERSION to $ENVIRONMENT"
echo "=========================================="

# 1. Validate
echo "Step 1: Validating..."
helm lint charts/ml-api
kubectl apply --dry-run=client -f k8s/overlays/$ENVIRONMENT/

# 2. Build & Push
echo "Step 2: Building image..."
docker build -t harbor.oilgas.internal/oilgas/ml-api:$VERSION .
docker push harbor.oilgas.internal/oilgas/ml-api:$VERSION

# 3. Scan
echo "Step 3: Security scan..."
trivy image harbor.oilgas.internal/oilgas/ml-api:$VERSION

# 4. Update manifests
echo "Step 4: Updating manifests..."
cd k8s/overlays/$ENVIRONMENT
kustomize edit set image harbor.oilgas.internal/oilgas/ml-api:$VERSION
git add .
git commit -m "Deploy $VERSION to $ENVIRONMENT"
git push

# 5. Sync ArgoCD
echo "Step 5: Syncing ArgoCD..."
argocd app sync ml-api-$ENVIRONMENT --prune

# 6. Wait for rollout
echo "Step 6: Waiting for rollout..."
kubectl rollout status deployment/ml-api -n oilgas-ml-$ENVIRONMENT --timeout=5m

# 7. Health check
echo "Step 7: Health check..."
kubectl get pods -n oilgas-ml-$ENVIRONMENT -l app=ml-api

echo ""
echo "Deployment complete!"
```

### 10.2 Quick Reference

```
# ========================================
# DEVOPS QUICK REFERENCE
# ========================================

# --- DOCKER ---
docker build -t image:tag .
docker push image:tag
docker-compose up -d

# --- KUBERNETES ---
kubectl get pods -n namespace
kubectl logs -f pod-name
kubectl exec -it pod-name -- /bin/bash
kubectl apply -f manifest.yaml
kubectl rollout restart deployment/name

# --- HELM ---
helm install name chart/ -f values.yaml
helm upgrade name chart/ -f values.yaml
helm rollback name 1
helm list -A

# --- ARGOCD ---
argocd app sync app-name
argocd app get app-name
argocd app history app-name

# --- VAULT ---
vault kv get secret/path
vault kv put secret/path key=value

# --- KUSTOMIZE ---
kustomize build overlays/production | kubectl apply -f -
kubectl apply -k overlays/production
```

---

*END OF DEVOPS TOOLS & OPERATIONS GUIDE*